ANURAG NAVIN BHANDARY

+91 8169370520 | bhandaryanurag@gmail.com | anuragbhandary.github.io | linkedin.com/in/bhandary-anurag | github.com/AnuragBhandary | Mumbai, India

SUMMARY

Software developer with a one-year internship on the founding team of an early-stage AI sports startup, working on **asynchronous services**, **event-driven pipelines**, and applied **LLM** systems. Primary stack is **Python** with **FastAPI**, **asyncio**, **Kafka**, **Redis**, **PostgreSQL**, and **Docker**, with an emphasis on throughput, ordering guarantees, and reliable failure recovery.

TECHNICAL SKILLS

Languages: Python (primary), **SQL**, **TypeScript**, JavaScript, **Java** (core language, data structures and algorithms)
Frontend: React, TypeScript, Vite, **TanStack Query**, React Hook Form, Zod, Tailwind CSS, **Playwright**, Vitest
Backend and APIs: FastAPI, **REST APIs**, **WebSockets**, API-key auth, rate limiting
Distributed Systems: Apache Kafka, **Redis** (pub/sub, queues), async workers, **idempotency**, retries, backpressure, event replay
Cloud and DevOps: Kubernetes, **Helm**, **Terraform**, KEDA, **Prometheus**, Grafana, **Docker**, GitHub Actions, **PostgreSQL**, AWS S3
AI and Media: Gemini, **Groq**, **ElevenLabs**, structured LLM output, OCR, **ffmpeg**

EXPERIENCE

Software Development Intern (Founding Team) | Fayble Inc. | Oakland, CA, USA | Jul 2025 - Jun 2026
AI sports-simulation platform

- Engineered the **audio pipeline** for an AI sports-commentary system, generating **dual-commentator** speech with **ElevenLabs** and stitching per-play clips into continuous broadcast output.
- Moved commentary text generation from **Gemini** to **Groq** and chose a one-emotion-per-play voice design over per-segment emotion stitching, keeping full-game audio generation fast and within free-tier limits.
- Automated archery match production with **ffmpeg**, synchronizing AI commentary, crossfades, **7.1 (8-channel) audio** with 1 kHz broadcast trigger tones, and structured per-play metadata into finished match videos.
- Integrated **Apache Kafka** into the platform's animation flow, replaying **19 games** and roughly **1,274 Kafka messages** in order through an ingestion path that blocked duplicates, then served game state through a **FastAPI** API.

PROJECTS

Distributed Task Scheduler and Job Queue | *Python, FastAPI, Redis, PostgreSQL, React, TypeScript, Docker*

- Designed a multi-service job platform with scheduled and immediate execution, **exponential-backoff retries**, dead-letter handling, **worker leases** with fencing tokens, **idempotency keys**, and persisted execution history.
- Sustained **2,000 jobs per minute** over a 50,000-job run while a worker was killed every minute: **zero** unrecovered failures, **p95 enqueue latency of 7 ms**, and no duplicate side effects across retries and crash recovery.
- Shipped REST endpoints and a **Celery-style Python client** with API-key auth and rate limiting, containerized at **94% test coverage**.
- Built a **React + TypeScript** operations dashboard; a **Playwright** worker-kill test confirmed **500/500 jobs** committed **exactly once**.

Real-Time Event Streaming Platform | *Python, asyncio, FastAPI, WebSockets, Redis, PostgreSQL, React, TypeScript*

- Wrote an **asyncio** event service that persists ordered events, serves state snapshots, and fans live updates across three instances through **FastAPI**, **WebSockets**, and **Redis pub/sub**.
- Held **p95 delivery latency at 5 ms** with **1,000 concurrent WebSocket clients** and 100,000 events (**5 million deliveries**), preserving monotonic sequence numbers.
- Added cursor-based reconnection, bounded replay, heartbeats, and **backpressure**, with zero duplicate or out-of-order events across **14,000+ reconnects** under client drops and instance kills.
- Built a **React + TypeScript** client that resumes from its cursor; in **Playwright** chaos tests every tab applied **1,502/1,502 events** once.

Cloud-Native Platform for the Job Queue | *Kubernetes, Terraform, Helm, KEDA, CloudNativePG, Prometheus, Grafana*

- Provisioned a five-node **Kubernetes** cluster with **Terraform** and **Helm**: HA **PostgreSQL**, Gateway API, and **Prometheus/Grafana** SLO burn-rate alerts.
- Autoscaled workers from 1 to 10 with **KEDA** on Redis stream lag; **20,000 jobs** at 400/s all completed exactly once, and with a node killed mid-run **117 jobs** were taken over with **0 lost or duplicated**.
- Traced an **OOM-killed** PostgreSQL primary to autoscaled connection pools; **PgBouncer** now caps it at **67 connections**.

EDUCATION

M.S. Computer Science | University of Texas at Arlington | Aug 2023 - May 2025
GPA 3.42/4.0 | Focus: Artificial Intelligence and Software Engineering | Arlington, TX

B.E. Electronics Engineering | University of Mumbai | Aug 2019 - May 2023
GPA 8.36/10 | Mumbai, India

Competitive Coding: LeetCode 450+ problems in Python and Java (data structures and algorithms) | leetcode.com/anuragb2901